Milestone 2: Data Understanding and Pipeline Setup

Student: Oisin Hickey | **Module:** Data Handling and Infrastructure for AI

GitHub Repository URL: https://github.com/OisinHick/AI-Hdip

Disclaimer: I used Google Gemini for formatting, spell checking, and table creation.

1. Project Definition

My project utilizes the Cybersecurity Attack and Defense Dataset (Attack_Dataset.csv) to develop a multi-class classifier which will predict the high-level threat category (i.e., Web Application Security, Cloud Security, Malware & Threat, etc.) from the text description and some basic metadata of an attack situation. The intended users of this system will be SOC analysts and/or incident responders needing to automatically sort incoming security tickets into the appropriate queue. To measure success criteria, the pipeline has to include 100% of rows read (i.e., rows_read = good_rows + bad_rows), ensure no duplication or post-incident data "bleed" between splits, and outperform the majority class baseline (accuracy = 3.98%).

Changes since Milestone 1 (based upon my EDA):

1. Deduplicate prior to split: As described in Milestone 1, I did split the 14,133 rows of Section 6 first then ran drop_duplicates() in Section 10. After reviewing the results of each split, there were 131 duplicate scenarios from the Train set leaking into Dev (47), Test (53), and Future Holdout (31). With Milestone 2, I am quarantining duplicate scenarios on (Title, Scenario Description) during validation prior to splitting.

2. Quarantine rather than drop rows with Unnamed: 15: In Milestone 1 I thought that I could simply drop Unnamed: 15 because I believed that it was merely an empty column. Upon completing my EDA, I realized that although there were 46 rows which contained values in Unnamed: 15, the values in those rows were a duplicate of the Tools Used field and caused all subsequent columns to shift over one position. There were also 82 rows containing pipe characters (|) in Attack Steps causing the MITRE ID to shift into the Impact column. All of these broken rows are now sent to bad_rows.jsonl.

3. Drop post-incident columns from X: Post-incident columns such as Impact, Detection Method, and Solution are only known after an investigation of an attack has taken place; therefore, using these as inputs for my model would cause temporal/target leakage (Slide 40 Lecture 5). I now only input pre-incident fields into the Column Transformer.

2. Data Source, Provenance, Governance and Ethics

The cyber security dataset is available to access via https://www.kaggle.com/datasets/tannubarot/cybersecurity-attack-and-defence-dataset. It was created by Tannu Barot & Savani Dhruv using an MIT license which allows for both academic and commercial use and modification.

Original Dataset Size: Raw file has 14,133 rows with 16 fields (SHA-256: 130e90c57e33d1791a3f2aad07855dcc80e7d6b0727ecd421376abc2550283d4) and is stored locally as read-only (chmod 444).

Limitations Regarding Ethics, Privacy, & Bias: No identifiable information or PII exists within this dataset. The IP address and URL used in each step of an attack are private RFC 1918 lab ranges (i.e., such as 192.168.1.x or localhost). Sampling/Representation Bias: This dataset consists of a collection of educational materials (such as OWASP training resources, MITRE ATT&CK, training labs) that have been developed primarily for educational purposes. As a result, almost all categories of threats include roughly 500 examples. In a real world Security Operations Center (SOC), it can be expected that phishing and web-based exploits would greatly exceed examples of Quantum Cryptography and Satellite Attacks.

3. Data Description

Each unit of analysis is a single documented cyber attack scenario that makes up each row of this data set. Of the 14,133 raw rows in total, there were 393 rows quarantined based on validation rules which includes; 262

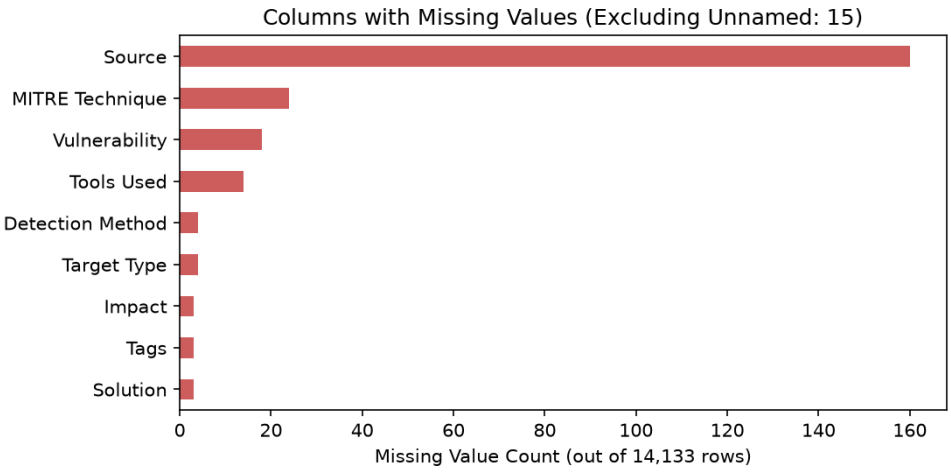
duplicated scenarios, 82 rows with shifted pipes (ie. The values in adjacent columns have been swapped); 46 rows in the unnamed: 15 column had an overflow value for the number of characters in that column; And 3 truncated rows. This left 13,740 valid rows to be analyzed.

The table below lists the main columns of this data set and their semantic roles in relation to how they are stored by pandas (slide 9 lecture 4) with missing values counts and an explanation of use.

Column	Semantic Role	Dtype	Raw Missing	Pipeline Role
ID	Record Identifier	int32	0	Excluded from features (identifier only)
Title	Short Text	string	0	Input feature (text)
Category	Target Label (y)	category	0	Target variable (59 classes after grouping <10 into Other)
Attack Type	Sub-category Label	string	0	Excluded from X (too close to target label)
Scenario Description	Unstructured Text	string	0	Input feature (TF-IDF + word/char length)
Tools Used	Semi-structured List	string	14 (0.10%)	Input feature (tool_count; missing imputed as Unknown)
Attack Steps	Procedural Text	string	0	Input feature (steps_word_count)
Target Type	Nominal Categorical	category	4 (0.03%)	Input feature (OneHotEncoder)
Vulnerability	Text / CVE	string	18 (0.13%)	Kept in cleaned table (imputed as Unknown)
MITRE Technique	Catalog ID (Txxxx)	string	24 (0.17%)	Excluded from X (curated with label)
Impact / Detection / Solution	Post-incident Text	string	3–4 each	Excluded from X (post-incident leakage)
Source	Provenance Category	category	160 (1.13%)	Excluded from X (curation source confounder)
Unnamed: 15	CSV Overflow Column	string	14,087 (99.67%)	Used to catch the 46 shifted rows, then dropped

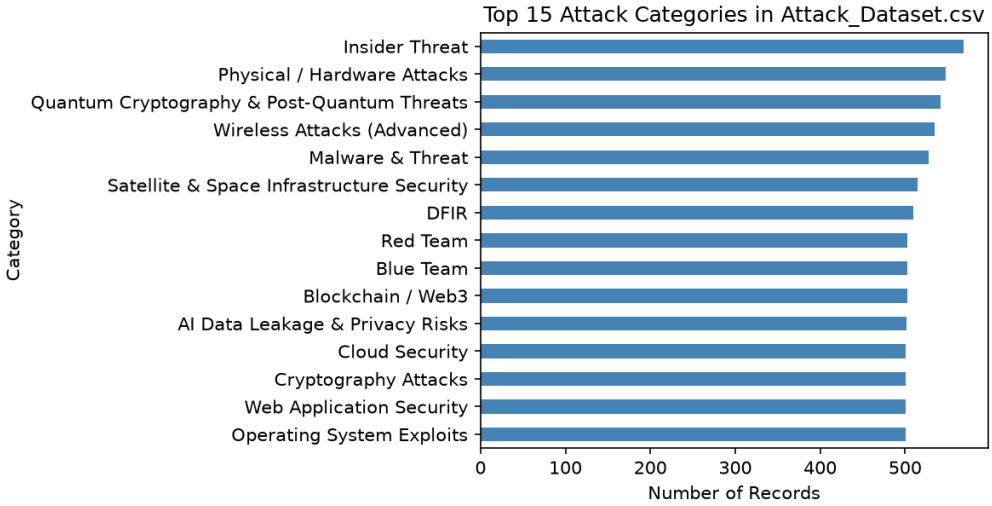
4. Exploratory Data Analysis (EDA)

4.1 Missingness (MCAR vs MAR vs MNAR): Outside of Unnamed: 15, all missing values were below 1.2%, however, they did NOT appear to be Missing Completely At Random (MCAR). The missingness by Category revealed 20 of the 24 missing MITRE Techniques had occurred within Digital Forensics (DFIR) since digital forensic procedures such as using tools like dd or FTK Imager for disk imaging are considered defensive actions rather than attack techniques (MAR). Similarly, there were 13 of the 14 missing Tool Utilization rows that have occurred under Insider Threat & Physical Attacks when no software tool has been utilized (MNAR/structural zero). Therefore, if I drop the rows based on having missing data then I will introduce biased results against both Digital Forensic and Insider Threat. Thus, I will impute these missing values with 'Unknown', and set tool_count = 0.



4.2 Target Variable and Class Balance: The majority of the 59 target categories can be grouped together by the main categories containing approximately 450-547 rows for most of them (after deduplication) and largest being

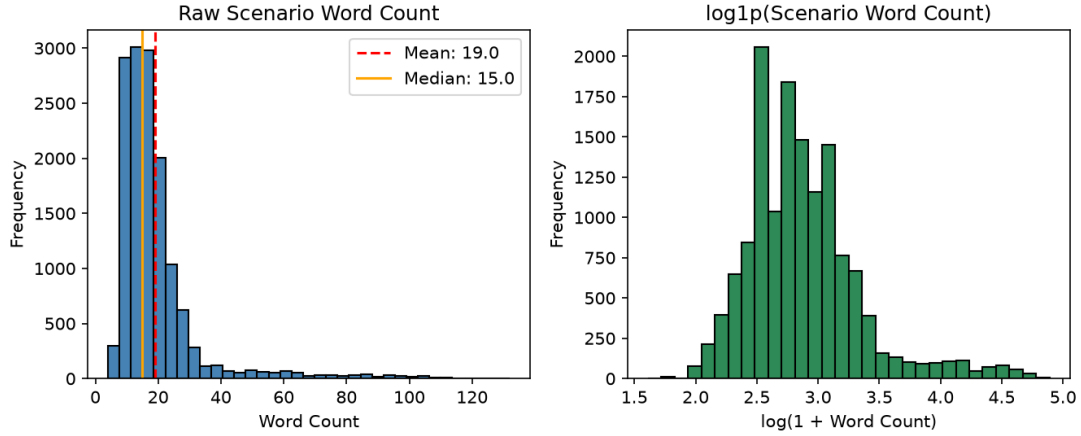
Physical/Hardware Attacks at 547/13,740 or 3.98% as the majoritry baseline. The remaining 4 rare automotive subcategories that contain less than 10 rows each will all be placed in "Other Specialized Threats" since they do not fit within their respective main categories. These categories also include a small number of subcategories that were created using an arrow to represent how one category is a subcategory of another (e.g., Browser Security -> XSS) which resulted in a smaller amount of data entries ranging from 10-60 per subcategory.

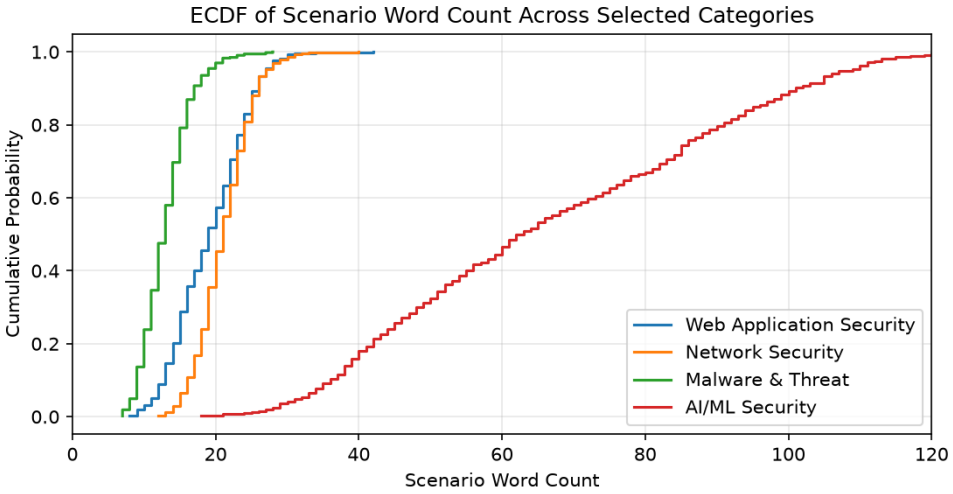


4.3 Numerical Summaries (Sensitive vs Resistant), Skewness, and ECDFs: Following Lecture 4 Slide 11 and Lecture 5 Slide 16, the table below compares mean and standard deviation (sensitive to outliers) with median and IQR (resistant) on the 13,740 valid rows.

Feature	Mean	Std	Median	IQR	1.5xIQR Fence	Outliers > Fence	Raw Skew	log1p Skew
scenario_len (chars)	128.60	91.82	104.0	64.0	240.0	880 (6.4%)	3.49	1.14
scenario_word_count	19.19	13.99	16.0	9.0	34.5	934 (6.8%)	3.53	1.17
steps_word_count	106.81	68.78	84.0	80.0	259.0	507 (3.7%)	2.09	0.31
tool_count	3.02	1.33	3.0	1.0	4.5	1,287 (9.4%)	2.40	0.20

As demonstrated by both the histogram and ECDF plots shown below, the data for the variable, scenario_word_count has a significant amount of right-skew bias (mean=19.2; Median=16.0; Skewness = 3.53). Upon reviewing the first 934 rows where the number of words was greater than 1.5 times the Interquartile Range (IQR) (i.e., 34.5), I noticed that virtually all the observations came from two categories: AI/ML Security (Median=63 words) and AI Agents & LLM Exploits (Median=41 words). If I had removed these data points, it would have been necessary to delete approximately 90% or more of the data from the AI/ML Security category. To prevent this, I retained those records but applied the transformation log1p(x) to reduce the skewness from 3.53 to 1.17 (steps_word_count reduced from 2.09 to .31 as well). The Chi-Square Test for Association between Category and Source resulted in a probability value less than 1e-100 (Cramér's V=.85); therefore, I am able to conclude that there is an association between Source and how data collections were carried out.





Traceability Summary (EDA Finding -> Pipeline Decision -> Limitation):

EDA Finding	Pipeline Decision	Limitation
131 shifted / truncated rows (Unnamed: 15 & pipe '|' shifts)	Quarantine to bad_rows.jsonl in validate_row()	SCADA/ICS loses 49 corrupted rows (452 -> 403)
266 duplicate (Title, Scenario Description) rows	Deduplicate in validate_row() before train/test split	Near-duplicates with slightly reworded text may remain
Missing values in DFIR (MITRE) and Insider Threat (Tools)	Impute 'Unknown' / 'Unmapped' and tool_count=0	Imputed token is a rough placeholder
Heavy right skew on word/tool counts (AI categories > 1.5xIQR)	Keep extreme rows; apply FunctionTransformer(np.log1p)	AI scenarios are still longer on average than other classes

5. Pipeline Architecture and Storage Design

The pipeline in pipeline.py is comprised of five major stages, including those referenced in Lecture 6 and Lab 6:

1. Ingest: Reads the original, read-only csv files Attack_Dataset.csv (which contains 14,133 rows), and cleans all additional white-space off the column header names.

2. Validate & Quarantine (validate_row()): Validates each row against the Data Contract for the project. All non-valid (or broken/duplicate) rows (i.e., 393 total rows that include 46 Unnamed: 15 overflow rows; 82 pipe-shifted rows; three truncated rows; and 262 duplicate scenarios) will be written to outputs/bad_rows.jsonl along with an associated reason tag. The remaining 13,740 valid rows continue through the pipeline.

3. Clean & Engineer Features (clean_and_engineer()): Corrects spelling errors for Category Names; aggregates rare Categories (< 10 total rows) into Other Specialized Threats; replaces missing Categorical Values with 'Unknown'; and computes scenario_len, scenario_word_count, steps_word_count, and tool_count. A copy of the clean Dataset is then saved to both outputs/attack_features_v2.parquet and outputs/category_summary.csv.

4. Stratified Split: Randomly splits the 13,740 valid rows into Training = 60%, Development = 15%, Testing = 15%, and a Future Holdout set of 10% (total of 1,374 rows) using a random state of 42.

5. Preprocess (build_preprocessor()): Builds a ColumnTransformer by first fitting it to only the Training subset of data (for the purpose of avoiding data leak); transforms the same features on the Development subset; and does likewise on the Testing subset.

Storage Design: The original data remains as CSV files originally downloaded from Kaggle (and verified by SHA-256 hash). The Cleaned Data is stored in Apache Parquet format attack_features_v2.parquet using pyarrow and Snappy compression - resulting in a reduction of size from approximately 17.4 MB down to roughly ~7.3 MB and maintaining all column dtypes. The bad_rows.jsonl file stores all non-cleaned rows so that each failed row may have a recorded reason. Both running pipeline.py and running pytest is accomplished automatically via CLI.

Downloading the original CSV file from Kaggle and reviewing all EDA plots within the provided Jupyter Notebook are manually completed tasks.

6. Pipeline implementation and validating evidence

Implementation of the pipeline has been made in pipeline.py file. The implementation was validated through five unit tests included in the test_pipeline.py file (valid rows, missing fields, wrong id's, unnamed: 15 shift and duplicate scenario.) running pytest and pipeline.py on the full dataset returns:

```
$pytest -v test_pipeline.py
5 passed in 0.89 s

$python pipeline.py --input data/raw/attack_dataset.csv --out outputs
Finished pipeline -> rows_read: 14133, good_rows: 13740, bad_rows: 393
Bad_rows breakdown: duplicate_scenario=262, pipe delimiter = 82, unnamed_15 = 46,
truncated_row=3
Sizes of split: train_rows=8244, dev_rows=2061, test_rows=2061, future_rows=1374
```

The accounting check also works since the data de-duplication occurs prior to splitting into train/test/dev/future sets therefore there are no duplicated scenarios present among test and train /dev/future.

7. Next-step plan (milestone 3)

Baseline model results:
In milestonetwo.ipynb, i compared a majority class baseline (DummyClassifier) to a leak-free LogisticRegression pipeline (log1p + StandardScaler on lengths/tool_count, OneHotEncoder on Target Type and TfidfVectorizer on Scenario Description) trained on the train set only:

Model	Accuracy on dev set	Test accuracy	Macro-F1 on test set
Majority class baseline (DummyClassifier)	3.98 %	3.98 %	0.13 %
LogisticRegression (leak-free pipeline)	71.47 %	70.94 %	42.08 %

Plan and limitation for milestone 3: although the accuracy of approximately 71% is achieved across the 59 classes, the Macro-F1 value is lower (approximately 42%) because the 31 smaller subcategories of arrows (e.g., 'browser security -> xss', which has 24 training rows) get misclassified as larger parent categories. To achieve an improvement at milestone three, I will:

1. Add the Attack Steps and tools used text to the TF-idf vectorizer together with Scenario Description.

2. Compare LinearSVC, Complement Naive Bayes and use class_weight='balanced' to improve precision on the smaller subcategories.

3. Perform evaluation using stratified 5-Fold Cross-Validation and confusion matrices.

Tools used: all of these were created with Standard Python libraries (pandas, scikit-learn, scipy, matplotlib) and AI coding assistant to assist in formatting the code/script/report; all results/data were manually confirmed by executing pipeline.py and pytest on the entire dataset.